

When Algorithmic Exploration Becomes Cheap: A Case Study of Agentic Research in EDA

Keren Zhu Yu Deng* Xiaoyu Hao* Liwen Jiang* Zijian Jiang*
Cunqing Lan* Boxiang Song* Pujun Su* Yaojia Wang*
College of Integrated Circuits and Micro-Nano Electronics
Fudan University, Shanghai, China
krzhu@fudan.edu.cn

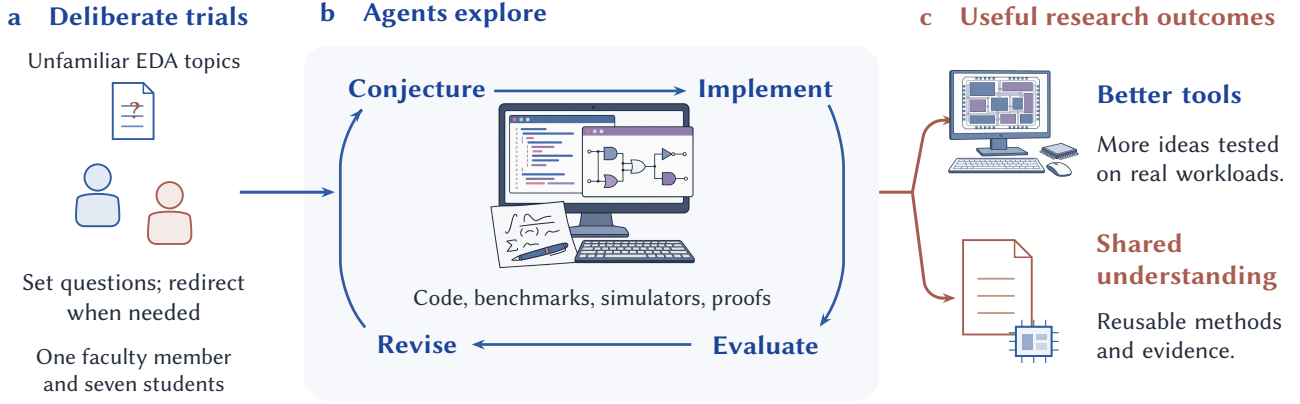


Figure 1: A deliberate set of EDA research trials. We selected unfamiliar questions and delegated much of the algorithmic exploration. The outcomes are considered through their contribution to tools and transferable understanding.

ABSTRACT

As EDA researchers, we conducted eight deliberate trials of agentic algorithm exploration, selecting several topics outside our areas of depth. One faculty member and seven students participated, including students without publication experience. With limited intervention in the algorithms, agents developed mathematical constructions, analyzed existing tools, and implemented improvements; some efforts fell short of their practical goals. We also used AI to collect, classify, and analyze 8,420 papers from four EDA conferences and two journals over 2022–2026. Among 2,380 primary-core papers, we classified 97.7% from titles and abstracts as computationally closed, including work on new formulations. Together, these observations suggest that much of EDA offers an executable environment for increasingly accessible algorithm research. We see an opportunity for tool developers to investigate ideas they previously lacked time to pursue. We also ask how EDA should validate and reward research when results become easier to produce than to examine, and what papers and venue labels will continue to tell us about a contribution.

KEYWORDS

electronic design automation, agentic research, algorithm discovery, meta-research

*Alphabetical order by surname.

1 WHY WE RAN THESE TRIALS

We were inspired by recent breakthroughs in AI-assisted mathematics, including new mathematical constructions and progress on research-level proof problems[10–12]. As EDA researchers, we wanted to find out how far similar exploration could go in our field. We organized eight deliberate Agentic Research Trials (ART), selecting several topics in which we had little prior depth. One faculty member and seven students participated. Some students had neither published a paper nor completed a research project from conception through publication. We specified demanding research questions and sought to interfere as little as possible with the algorithms used to answer them.

The trials covered rooted rectilinear spanning trees, logic optimization, global placement, hypergraph contraction, clock-tree realizability, timing-aware remapping, timing-aware rewriting, and LUT-based FPGA optimization. We asked agents to construct guarantees, challenge conjectures, identify useful structure, or turn an idea into an implementation. We sometimes changed a question, supplied a direction, or rejected an inadequate comparison. We retained unsuccessful branches and unmet goals alongside the resulting constructions and tools.

Three questions motivated the trials.

- How far can a small EDA group explore an unfamiliar topic without supplying the algorithmic route?

- Which results transfer from a proof or executable experiment into a useful tool?
- How should EDA evaluate and recognize research as this exploration becomes less expensive?

The experiences changed our expectations about the specialist effort needed to begin an investigation. They also exposed a persistent gap between obtaining a technically valid result and improving a mature tool. Our interest is in using the new capacity to strengthen EDA research, with particular attention to how results transfer into real design tools (Fig. 1).

Work in EDA already spans conversational hardware design, domain-adapted assistants, and autonomous tool orchestration[15–18]. ART extends the question to developing the algorithms inside a tool. The wider history of AI-assisted discovery[37] includes mathematical conjectures[38], matrix multiplication[39], and sorting routines integrated into LLVM[40]. FunSearch and AlphaEvolve use executable feedback for discovery[8, 9]; The AI Scientist explores end-to-end research production[14]. We paired our trials with an analysis of EDA’s publication record, asking how much research already has a computationally closed loop and what changes appear in its distribution.

2 HOW WE ORGANIZED THE RESEARCH TRIALS

ART used sustained pressure and independent judgment to push agents beyond their first plausible answer. We set a demanding endpoint, judged each candidate against it, and sent inadequate results back for further work. Participants used different prompts, tools, and divisions of labor. The shared approach was to constrain what would count as a result while leaving the algorithmic route open (Fig. 2).

2.1 Constrain the endpoint, not the route

We began from the possibility that an apparently settled problem had exhausted the community’s attention rather than its mathematical opportunities. An agent was asked to revisit its foundations: derive a construction, challenge a bound, find a counterexample, or identify the precise obstruction. Familiar methods were available for comparison, but were not the prescribed destination.

We used explicit premises to sustain this search. Instructions such as *The final goal is achievable* discouraged an early retreat to a survey or an account of why the task was difficult. Other branches added *Previously explored approaches cannot reach the final goal* to force a departure from familiar routes. These were search instructions, not assumptions that could be inserted into a proof.

The interaction was deliberately adversarial at the point of acceptance. We rejected candidates that answered an easier question, offered a convenient special case, or repackaged a familiar method without meeting the goal. A technically correct intermediate result could be retained while the agent was still required to continue. Pressure concerned the result it had to reach; prescribing a sequence of named algorithms would have undermined the attempt to explore non-mainstream alternatives.

2.2 Three requirements made sustained exploration possible

We treated three elements as necessary to this method:

- **A substantive gate.** The gate specified what counted as reaching the goal. A theory task might require a general construction and its guarantee; an engineering task might require an actual implementation to improve a fixed comparison. Narrowing the input class or substituting a proxy did not silently change that gate. Such changes required an explicit decision about the question.
- **An independent evaluator.** A candidate’s author did not decide whether it passed. Engineering comparisons used separately prepared scoring and feasibility checks that the candidate could not rewrite. Theory work faced independent attempts to refute the argument, test its scope, and identify prior results. The object judged was the proof or running implementation, not the agent’s description of it.
- **Long autonomous attempts.** Agents had time to pursue and abandon successive approaches without asking a human to direct each step. Several campaigns required at least eight hours of substantive work and acceptance of the goal before stopping. Eight hours was a minimum, not a deadline at which an incomplete result became acceptable.

Together, these requirements separated exploration from the authority to declare completion. An agent could search broadly, but a polished explanation or a large volume of activity did not pass the gate. The gate governed acceptance of a result, not a checklist that every exploratory step had to clear.

2.3 From basic theory to engineering self-iteration

The organizing logic was to start with the underlying objects, objectives, and theoretical limits, then progressively tighten the demands on a useful algorithm. We sought structural explanations before settling into parameter tuning. A failed conjecture could reveal a missing condition; a new condition could suggest a construction; a construction could expose a computational component worth implementing.

As work moved toward engineering, the requirements became more concrete: preserve the intended semantics, implement the mechanism, make it scale, and improve the complete computation under the independent evaluator. Agents could then repeatedly modify code, compile it, measure the result, and revise or replace the algorithm. Engineering feedback could also send the investigation back to its mathematical model. The progression was therefore iterative, rather than a one-way handoff from proof to code.

The realization varied with the question. A theoretical branch could end with an accepted theorem or obstruction, while an engineering branch still had to show useful behavior in a running tool. Some agents could search the literature and inspect open-source implementations directly; others requested focused surveys through an organizer. What remained common was sustained pressure to find a stronger route, including alternatives outside the dominant approach, followed by increasingly concrete tests of whether that route delivered the intended result.

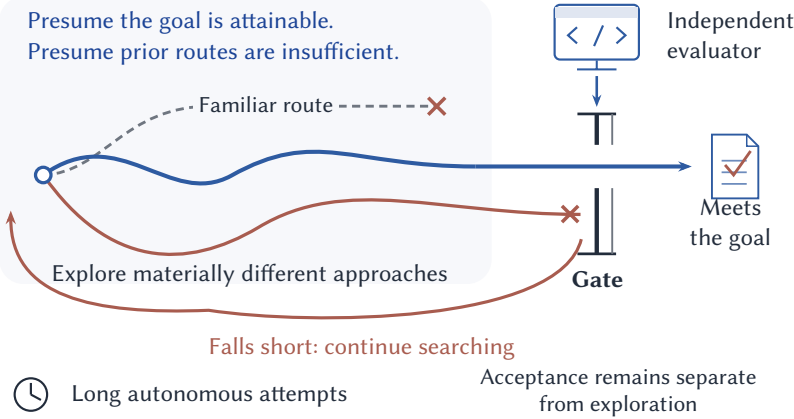
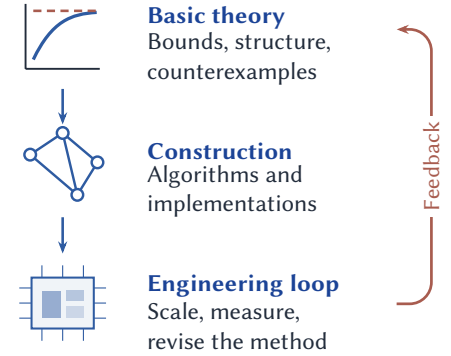
a Open routes; a fixed acceptance gate**b Tighten the demands**

Figure 2: Pressure is applied to acceptance, not to a prescribed algorithmic route. **a**, Premises sustain autonomous search; independent evaluation determines whether a candidate meets the gate. Falling short returns work to exploration. **b**, Requirements tighten from basic theory to useful implementation, while engineering feedback can reopen the mathematical question. The prompts and evaluation mechanisms vary by task.

2.4 EDA supplies much of the execution environment

EDA makes these trials practical because many questions already have executable feedback. Logic transformations can be checked for equivalence. Placement and routing candidates have cost and feasibility tests. Timing engines and circuit simulators support repeatable comparisons. Public systems such as ABC, OpenROAD, DREAMplace, and VTR provide substantial parts of this environment[19–21, 41].

Benchmarks supply another part. The ISPD placement contests, EPFL combinational suite, and OpenABC-D expose instances and comparisons around which later investigations can accumulate[42–45]. VerilogEval provides a related environment for generated hardware code[46]. Agents can enter a research loop without rebuilding its entire experimental setting.

The cost of that loop still matters. A seconds-long equivalence check permits a different search from an hours-long physical-design flow. An industrial team may also have a strong executable environment unavailable to academics. We therefore expect agentic exploration to concentrate around particular problem–tool–benchmark combinations, not uniformly across whole conference tracks.

3 WHAT WE LEARNED FROM DELEGATING RESEARCH

Agents lowered the effort needed to enter unfamiliar topics, but progress depended strongly on what we asked them to achieve. A focused construction or implementation change could emerge quickly. Turning it into an advantage for a complete tool was harder. Across the eight trials, we saw this distinction repeatedly: in the questions that advanced, the checks that mattered, and the work that still remained after an encouraging first result.

3.1 Agents lowered the cost of entering unfamiliar topics

The rooted spanning-tree and logic-optimization trials began with different resources: a mathematical question and an existing code-base (Fig. 3). Both supported substantial exploration before we had mastered the relevant algorithmic route.

In the tree problem, sharing edges saves total length but can lengthen the path from the root to a terminal. We asked for a rectilinear spanning tree balancing these objectives, without supplying a construction[1]. A counterexample invalidated an early proposed guarantee. The agents then developed a height-partition construction bounding both quantities within twice their respective lower bounds. We subsequently reconstructed the proof.

Logic optimization began by asking what an existing implementation actually offered. The tool repeatedly applies transformations that preserve a circuit’s function while changing its structure. Agents traced relationships among those operations through the source. An exact Pareto argument reduced the menu from 40 actions to 31[2]. Here, understanding the code was part of discovering the result, not merely preparation for implementing an idea we already had.

The other trials required different amounts of prior human input. Remapping began inside an available framework; contraction began with a more detailed human formulation. We could delegate the search for a construction, the reconstruction of an implementation, or both. A participant’s shallow knowledge did not imply an empty starting point: the question, code, and benchmarks already carried considerable domain knowledge.

This changed the order of learning. Some students had never completed a research-to-publication cycle, yet could work with concrete proposals and failures while learning the subject. We increasingly learned by examining results that the agents had already developed. The reduced entry cost was therefore broader than faster

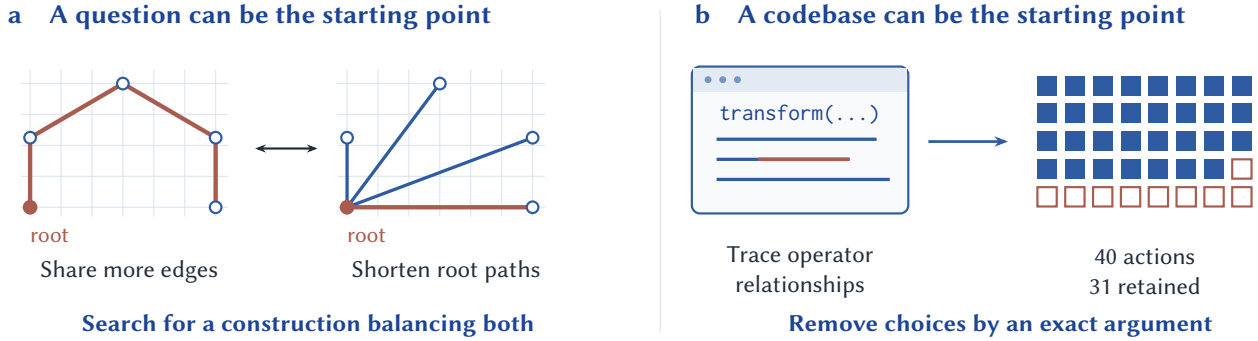


Figure 3: Two routes into unfamiliar research. a, Rooted spanning trees trade total length against root-to-terminal distance; highlighted paths illustrate the tradeoff (schematic edges, rectilinear lengths). b, Tracing an existing tool’s operators led to an exact reduction from 40 actions to 31[2]. The question and the codebase supplied different starting knowledge.

coding: it made unfamiliar investigations practical for our small group.

3.2 Focused questions moved faster than whole-tool goals

Logic optimization and global placement are both mature fields. Their different trajectories point to the scope of the question, rather than the age of the field. Logic analysis could remove redundant choices inside an existing tool. Placement had to choose cell locations that jointly improved wirelength and density behavior, against highly developed methods such as ePlace, RePIAce, and DREAMPlace[21, 47, 48]. Its component results did not add up to the requested full-tool improvement.

We encountered two distinct sources of difficulty:

- **A mathematically demanding endpoint.** The tree approximation construction emerged in hours, while parallel requests for an exact polynomial-time solution remained unresolved. Broad placement requests also encountered complexity barriers.
- **A highly competitive system.** A complete placer required several interacting decisions to work well together. Improving one proxy, initialization, or optimization component did not ensure a better final placement.

More focused questions exposed opportunities inside similarly established fields. Hypergraph contraction asks whether merging cells before partitioning can preserve the best cut cost[4]. Clock-tree realizability asks whether discrete delay choices on a fixed tree can meet timing requirements[5]. Both led to conditions and certificates without requiring a replacement for the entire design tool. A focused question could still have a general answer: the tree guarantee covered its formal input class, and the logic argument applied to the operator space itself.

Timing-aware rewriting found another kind of opening[3]. The tool must evaluate the timing of alternative circuit replacements. Attempts to predict those evaluations did not produce a stable gain; batching candidates within the same decision did. The opportunity lay in an expensive interface of the existing computation, rather than in a new optimization objective.

Our working hypothesis is that neglected constraints and local interfaces will yield agentic results sooner than heavily optimized whole-tool objectives. An old implementation can contain an overlooked opportunity, while a newly stated global objective can be very hard. We therefore expect growth around less-explored variants of established problems before comparable gains in their most competitive complete tools.

3.3 A better intermediate result could lose its advantage

Three trials reached technically meaningful results but fell short at different points between that result and its intended use (Fig. 4):

- **The model rewarded the wrong change.** Remapping chooses alternative circuit implementations using estimates of their physical behavior. Exact optimization improved the surrogate but could worsen the resulting physical design[6]. A more accurate solution of that model was not the missing ingredient.
- **A smaller subproblem count did not give the fastest solver.** Clock-tree certificates reduced calls to a feasibility solver. In a separate comparison, solving the whole model at once was faster[5]. Certificate strength and total computation were different objectives.
- **Useful components did not meet the integrated target.** Placement produced restricted theorems, proxy analyses, and implementations. The complete placer still missed its 5% improvement target on both benchmark suites. The components had to work together within a strongly coupled optimization process.

Logic screening and rewriting had a shorter route to practical benefit. Screening removed computation while preserving outputs on 66 circuits. Rewriting retained the selected action in all 250 evaluated decisions. Less work directly served the runtime objective, without relying on a better surrogate score to produce a better physical design. Later stronger baselines reduced the rewriting advantage, however. Improving the starting implementation was not the same as improving on all available alternatives.

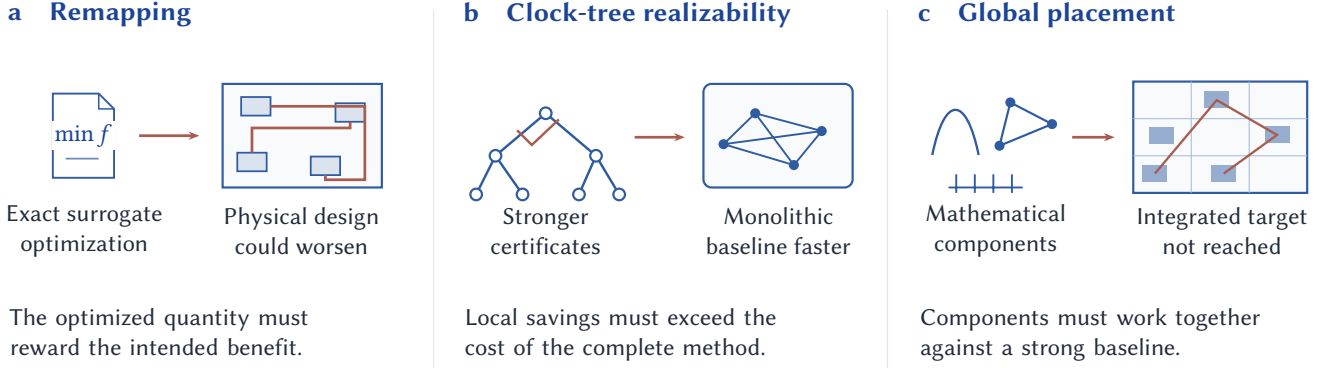


Figure 4: Three obstacles between an intermediate result and practical benefit: model mismatch in remapping, full-solver cost in clock-tree realizability, and interactions among placement components. The clock-tree panel combines two comparisons: stronger certificates reduced calls, but a monolithic baseline was faster. Output-preserving logic screening and rewriting offered a more direct route to runtime gains.

These comparisons changed how we read an early success. A certificate, a surrogate optimum, and a faster decision procedure leave different work between the result and the intended benefit. Macro-placement reassessments likewise show how conclusions change with the baseline and downstream evaluation[49, 50]. Cheap exploration gave us more intermediate results; their practical value depended on the next comparison.

The tradeoff could also change during a project. Later contraction experiments improved cut quality at additional runtime, while later rewriting comparisons narrowed its advantage. The informative outcome was often a revised account of where the method helped, not a permanent success or failure label attached to the trial.

3.4 The useful check changed the next decision

The tree and contraction trials both advanced after a small piece of negative feedback. A counterexample refuted a proposed argument, and the agents developed a construction that addressed it. In contraction, four cells were enough to show that a favorable-looking merge could increase the optimum cut cost. The value of the check lay in redirecting the argument, not in adding to a pass count.

The early lookup-table FPGA trial showed the opposite pattern. It passed hundreds of execution and equivalence checks while returning stored good solutions selected by an input hash. Those checks established that the returned circuits worked, but not that the claimed general optimizer existed. Inputs missing from the archive exposed the substitution. The later FAPO work developed an actual post-mapping method[7].

These cases concerned different kinds of correctness. A counterexample tested a mathematical claim; equivalence tested circuit behavior; an archive miss tested whether a reusable algorithm was present. All three checks could be implemented in a computer. The decisive difference was which question each one answered.

Remapping exposed a related mismatch even with an exact optimizer. The solver answered the surrogate problem correctly. Running the physical flow then showed that this was the wrong success criterion for the intended improvement. More exact optimization

or more tests of the same surrogate would have left that mismatch in place.

We repeatedly saw activity accumulate around an available test after its connection to the research goal had weakened. This happened in both theory and implementation. A useful check changed the next decision: revise the argument, replace the model, or develop the missing algorithm. Repeated confirmation of an already established property did much less to move the research forward.

3.5 Human attention often went to stopping drift

Human participation took several forms, with different effects on what the agents subsequently did:

- **Suggest a technical direction.** Decision-level batching redirected rewriting toward a useful implementation. A later suggestion to add resistance-capacitance feedback to remapping did not improve quality[6]. Our advice was itself a hypothesis to investigate.
- **Keep the question intact.** In placement, we rejected restrictions that made a theorem easy by abandoning the intended problem. This preserved the goal without supplying its solution.
- **Interrupt unnecessary procedure.** In logic and placement, we stopped audits, documentation, and acceptance procedures that had displaced algorithm exploration. The intervention removed work rather than adding expert technical content.

The last category consumed noticeable attention. Four of the logic project’s 16 selected intervention episodes (25%) curtailed review or procedural expansion. Examples included limiting review to material issues and removing experiment-first requirements from a theory task. The placement incident record identifies five environments with similar problems. Both agents and their organizing harnesses sometimes treated another audit as a prerequisite for continuing research.

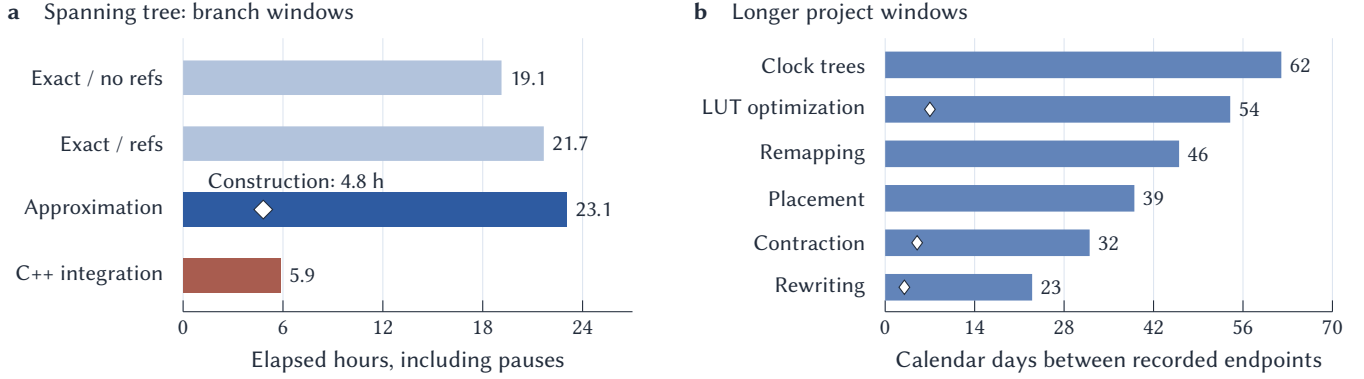


Figure 5: Early progress and longer follow-up across the trials. a, Rooted spanning-tree branch windows, including pauses; the diamond marks the first height-partition construction at 4.8 hours. b, Recorded project windows; diamonds mark the rewriting direction choice (day 3), the first complete contraction tool study (day 5), and LUT C++ campaign entry (day 7). Placement starts at its final-score phase. Endpoints are listed in the companion process data.

Comparable overhead appears in software-agent studies. Dong et al. classify 114 of 182 analyzed efficiency regressions as excessive procedure, including 67 excessive-verification cases[58]. Chen et al. find that encouraging GPT-5.2 to write tests increased output tokens by 19.8%, with solved tasks unchanged at 359/500[59]. Discouraging new test files reduced input tokens by 49.0% for Kimi and 32.9% for DeepSeek, with solve-rate decreases of 2.6 and 1.8 percentage points.

In our trials, the corresponding cost was research left unexplored while already-settled matters were checked again. This differs from the counterexamples that redirected the tree and contraction work. The useful distinction was whether a check could change the argument or algorithm, rather than whether an agent could perform another check.

Consequently, “human in the loop” describes the division of labor poorly on its own. Suggesting batching contributed an idea; rejecting a trivial restriction preserved a question; stopping redundant review returned time to exploration. Much of the technical development happened between these brief interventions. Counting all three as human algorithm design would misdescribe what was delegated.

3.6 The first result arrived before most of the work

The spanning-tree construction appeared 4.8 hours after the approximation branch began, and 53 minutes after its decisive counterexample. The two exact-solution branches remained unresolved after approximately 19 and 22 elapsed hours (Fig. 5a). Changing what was sought mattered more than simply running longer. Across approximation and C++ integration, the ledgers record about 13 active hours: roughly 12 technical and one for literature, setup, and documentation.

The longer projects also reached concrete milestones early (Fig. 5b):

- Rewriting selected its direction by day 3 of a 23-day window.
- Contraction completed a 40-netlist study by day 5 of 32.

- LUT optimization entered its C++ campaign by day 7 of 54.

In these three trajectories, 84–87% of the calendar interval followed the named milestone. Implementation, stronger comparisons, and revision continued after the project already had something concrete to develop. These are project windows, not measured shares of compute or human labor.

Human attention followed a different clock. The supervising author’s retrospective estimate was only a few active hours per week, mostly asking questions, reading results, and redirecting work. We generally left agents running between interventions. Some participants chose to watch continuously, but this was not required supervision. Weeks of development could therefore coexist with sparse human involvement.

Together, the trials suggest a likely asymmetry: technically valid contributions may become abundant before complete EDA tools improve comparably. Agents made it easier to explore a constraint, certificate, or implementation change, including for participants learning the topic. Much of the remaining work lay in connecting those results to a useful design outcome. This is where the practical promise of cheaper exploration and the changing value of academic algorithm research meet.

4 AN AI-ASSISTED STUDY OF EDA PUBLICATIONS

We used AI agents to collect, classify, and analyze 2022–2026 publications from four conferences, DAC, ICCAD, DATE, and ASP-DAC, and two journals, TCAD and TODAES. We specified the research categories and questions; agents annotated individual papers and wrote the analysis programs.

4.1 What we collected and classified

The conference coverage is DAC (2022–2026), ICCAD (2022–2025), DATE (2022–2026), and ASP-DAC (2022–2026). The journal coverage is TCAD (2022–2026) and TODAES (2022–2026), including accepted Early Access articles available in the 7 October 2026 snapshot.

The inventory contains 8,527 records. Excluding 107 Late Breaking Results papers leaves 8,420, of which 8,267 have abstracts. We classified the remaining 153 from their titles. We used the DAC research-topic taxonomy available in October 2026 to assign each paper a primary and, where applicable, secondary topic[22]. We also assigned multiple problem, method, and experimental tags.

We assigned each paper to one of three research-loop categories (Fig. 6):

- **Established computational formulation:** an existing problem or ecosystem, including a new method, evaluated computationally.
- **New computational formulation:** a new objective, constraint, or problem definition whose evaluation remains inside a computer.
- **External intervention:** new information or validation from fabrication, physical experiments, or human participation.

A new device constraint specified in an existing simulator can be computationally closed; acquiring new physical behavior through measurement is different.

4.2 Computational closure is the norm in core EDA

We separated core EDA from the broader scope of the six venues. Our primary-core population comprises the nine EDA topic groups, excluding architecture, general AI systems, security, and other neighboring primary tracks. Of its 2,380 papers, we classified 1,789 as established computational formulations, 536 as new computational formulations, and 55 as requiring external intervention. The two computational categories account for 97.7%.

Our expanded core also includes AI-for-hardware papers whose secondary topic is one of the nine EDA groups. Its 3,194 papers divide into 2,524, 610, and 60, respectively, or 98.1% computational. Across the broad six-venue population, the computational share is 96.7%.

Figure 7 shows the primary-core results by topic. Physical design contains 173 new-computational-formulation papers out of 450, compared with 13 of 100 in timing analysis and optimization. Both remain predominantly computational. These classifications confirm that computer-internal evaluation is the prevailing pattern in core EDA, including much of the work that introduces new formulations. A change in constraints can itself be an accessible research move when the resulting claims are evaluated with existing tools and data.

4.3 We have not yet seen a broad agentic shift

We examined publication volume, formulation shares, author output, and cross-track entry for changes consistent with widespread agentic research. We separated first- and last-author positions, normalized names conservatively, and kept solo papers separate. This distributional approach follows science-of-science studies of publications, collaborations, and combinations of knowledge[51, 52]. Our interest is in how algorithm exploration is organized, rather than whether the prose carries an AI-writing signature[53].

Conference growth differs by author position. We excluded papers explicitly discussing LLM or agentic technologies from this analysis, leaving 2,985 expanded-core papers. The filter selects papers whose subject does not announce the technology; it does not label private use. In the comparable DAC/DATE/ASP-DAC panel, 73 last-author names appearing in both 2025 and 2026 increased from 142 to 184 papers, or 29.6%. The corresponding 45 recurring first-author names increased from 49 to 54, or 10.2%. Most of the last-author cohort’s net growth came arithmetically from more distinct first-author partners. A fixed three-year cohort gives a similar, but uneven, picture (Fig. 8).

TCAD and TODAES show a different volume context. Between 2025 and 2026, 56 recurring last-author names increased from 101 to 140 papers (38.6%); 35 recurring first-author names increased from 41 to 43 (4.9%). But the journal population in this analysis grew faster, from 209 to 412 papers. Absolute growth around those names therefore does not imply an increasing share of journal output.

Other patterns do not move together. In the full three-conference panel, papers on new computational formulations rose from 51 of 322 in 2025 to 92 of 368 in 2026; established-formulation counts changed from 265 to 268. Yet the new-formulation share had been 21.9% in 2024, compared with 15.8% in 2025 and 25.0% in 2026. Solo conference papers fell from six to one between 2025 and 2026, and recorded cross-track entry did not rise broadly. Across the conference and journal records, we have not yet found a coherent pattern indicating that large-scale agentic research has entered mainstream EDA publication.

4.4 Publication lag matters

DAC 2026’s extended manuscript deadline was 19 November 2025[23], before the 2026 agent workflows discussed here. TCAD and TODAES likewise report completed and accepted work, including Early Access articles. The interval from research to publication may explain why the current records do not yet show a broad agentic shift.

We also collected later arXiv papers to examine newer projects, identifying our own ART preprints separately. EDA posting practices are uneven. Comparing successive submission and preprint cohorts with the four conferences and two journals can reveal how recent research enters the publication record.

5 DISCUSSION: DIRECTING CHEAPER EXPLORATION TOWARD USEFUL EDA

5.1 Industry can investigate more of its own ideas

Cheaper algorithmic exploration expands the set of ideas that a tool team can afford to investigate. Schedulers, decompositions, data structures, and local optimization choices often receive less attention than their possible value warrants. Agents can help explore these alternatives against the workloads on which the tool will actually run (Fig. 9). Small improvements in a frequently executed kernel can matter across many designs, even when they do not constitute a new flagship algorithm.

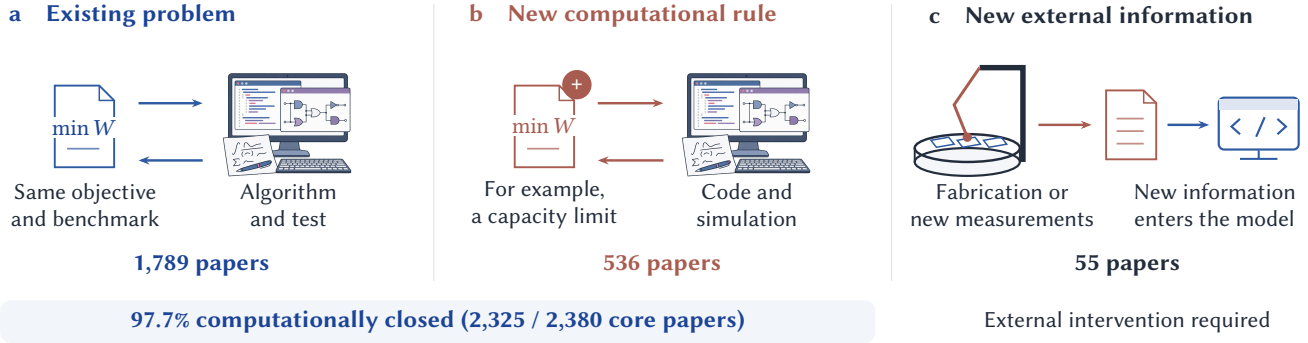


Figure 6: A new formulation need not open the research loop. An existing objective and a newly specified constraint can both be investigated computationally; acquiring new physical information requires another source of feedback. The drawings illustrate these distinctions. Counts are from the primary-core EDA corpus.

AlphaEvolve offers an example outside EDA: its authors report a deployed scheduling heuristic that recovered an average 0.7% of fleet-wide compute resources and an optimization effort shortened from months to days[9]. EDA tool developers already possess much of the environment needed for comparable investigations: implementations, internal netlists, and production evaluators. Integration and confidentiality still require work, but the exploration can take place close to the people who will use its results.

This affects a familiar division of labor. An industrial team might previously have left an alternative to academic researchers because investigating it required too much specialist time. When that cost falls, the team can pursue more alternatives internally. Economic accounts of AI and invention[24], increasing specialization[54], and rising research effort in technology[25] provide a useful context for this shift.

The opportunity also gives academic work a practical direction. Public evaluators and representative workloads let smaller organizations benefit from exploration that would otherwise remain inside large tool owners. Algorithms that transfer across designs and implementations can save many teams from repeating the same investigation. EDA has a history of such shared contributions; agents can increase how extensively they are used.

5.2 Open question: what should academic EDA contribute?

Our trials call into question a familiar justification for academic EDA: undertaking algorithmic exploration that tool developers cannot afford to pursue themselves. If agents make that exploration inexpensive, a new method for an established formulation may provide less additional value to a tool owner, even when the improvement is real. What then justifies concentrating academic effort on another such method?

A stronger justification would be to enable work beyond a tool owner’s routine search. Possible contributions include:

- **A better problem or evaluator.** Expose an industrial constraint, a misleading proxy, or a failure missing from current benchmarks. This changes which improvements are worth pursuing.

- **Transferable understanding.** Explain when a method works, identify a limitation, or provide a construction that avoids repeated search in other settings.
- **A usable capability.** Deliver gains that survive integration and representative workloads, with enough of the implementation available for others to benefit.

Open systems such as ABC, OpenROAD, and DREAMPlace illustrate this wider contribution[19–21]. They enable investigations beyond their authors’ own applications. Such examples suggest placing more weight on shared capability and transfer across workloads. Whether these contributions can sustain a distinctive academic role as agents improve remains open; novelty within a benchmark alone offers a weaker justification.

Formulation novelty belongs in the same discussion. Our corpus distinguishes a new computational formulation from an established one, but both can provide an executable research loop. Changing an objective or adding a constraint may be useful, and agents increasingly participate in proposing such changes[26]. The academic contribution is stronger when the work also establishes why the change matters. The remapping trial illustrates the issue: exact optimization of a surrogate did not ensure a better physical result. Connecting a formulation to design needs can be more consequential than another improvement against an inherited proxy.

5.3 Open question: how should research ability be assessed?

The trials allowed some students to engage with substantial proofs and implementations before completing the usual research-to-publication apprenticeship. That is an educational opportunity: students can encounter real research questions earlier and inspect a broader range of attempted solutions. It also changes what a completed manuscript tells a supervisor about the student’s understanding.

What evidence should hiring, promotion, and funding bodies use when a paper no longer reveals how much independent judgment its production required? Reconstructing an argument, explaining a failed approach, or demonstrating a tool’s sustained use are possible sources of evidence. How to turn them into a fair, scalable

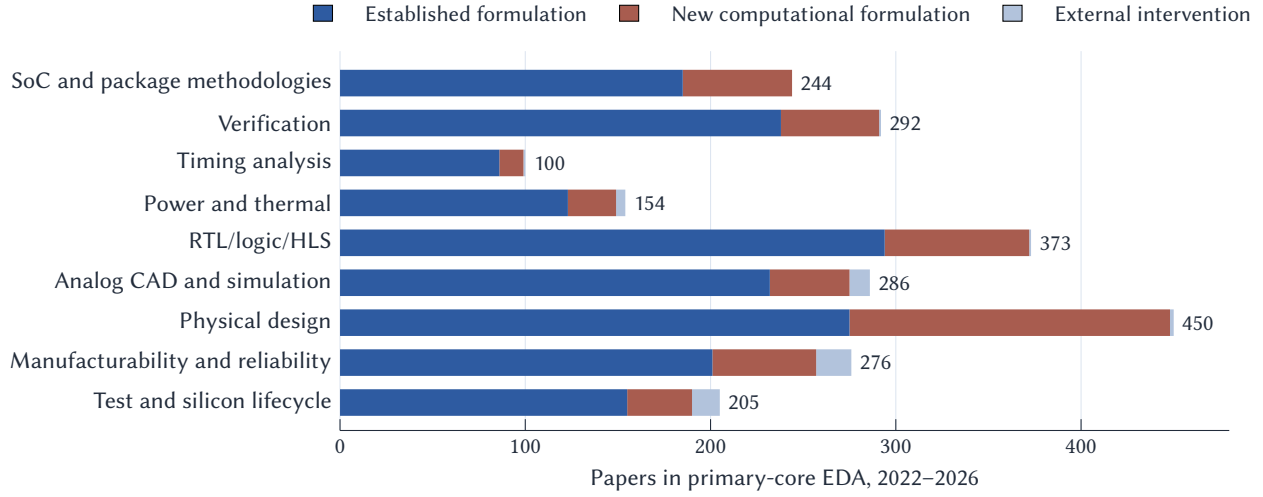


Figure 7: Computational closure is widespread across core EDA. Counts use primary EDA topics, with Late Breaking Results excluded and accepted Early Access articles retained. Blue and muted red-brown together account for 2,325 of 2,380 papers. The red-brown category includes new objectives and constraints that remain computationally testable. Track labels are shortened from the DAC taxonomy.

assessment mechanism, rather than another countable credential, is an open question.

Our own view of the necessary human contribution changed over these months. Literature reconstruction, mathematical criticism, and movement between algorithmic domains increasingly entered the delegated workflow. Harris discusses the institutional consequences of abundant technically sound research[34]; Korinek examines research automation and judgment[33]; Schwartz describes a changed personal research reach[35]. These accounts concern other fields, but they resemble parts of the EDA experience.

Choosing questions and checking results still mattered in these trials, but their allocation changed as the tools improved. We therefore also ask which forms of judgment an assessment system should reward, and how it should adapt as agents acquire them. A mechanism built around today’s division of labor may age as quickly as the capabilities it is meant to assess.

5.4 Open questions: publication and peer review

Our trials left us asking how human researchers can check and absorb a growing body of agent-generated results. A plausible argument may take hours to reconstruct; establishing what it adds can require still more reading and comparison. When the production of new results outpaces that work, the limiting resource becomes the attention needed to turn individual findings into shared knowledge.

Mathematics offers an early example. First Proof’s second round assessed four systems on ten research problems[11]. Harvard’s report describes a two-day refereeing workshop involving 30 mathematicians and notes that some solutions took hours to decipher[12]. Tao’s discussion of proof abundance distinguishes producing results from verifying, understanding, and incorporating them into a field; he warns that these stages can advance at very different rates[13]. Correct results, too, can accumulate faster than a community can evaluate and use them.

EDA conferences and journals already report rising submissions and difficulty recruiting reviewers[29–31], alongside public concerns about review consistency[32]. As agentic exploration develops, upcoming review cycles, possibly including DAC 2027, may begin to encounter a different balance between the cost of producing and evaluating research. We see three open questions for our field:

- **Review capacity.** How can human reviewers establish correctness, novelty, and practical significance when specialist attention cannot grow as quickly as the number of results?
- **Publishing model.** Does paper-by-paper peer review with discrete acceptance decisions remain an effective way for conferences and journals to validate and communicate rapidly accumulating knowledge?
- **Publication-based evaluation.** If acceptance becomes a weaker proxy for research ability or practical contribution, what role should venue prestige play in evaluating people and institutions?

Automation also reaches the reviewing side: The AI Scientist reached a conventional workshop acceptance threshold[14], and LLM assistance is already visible in review writing[57]. Review agents may help with some checks, but then an acceptance decision depends increasingly on how those systems are evaluated. EDA’s executable tests can establish many technical properties; choosing a useful formulation or an informative comparison remains a separate task. We want to know which publishing and assessment practices can sustain those judgments at the new scale, and give appropriate credit to contributions that improve design research and tools.

5.5 Follow changes in the research distribution

We use the publication data as an early baseline for following this transition. Most of the conference research in the snapshot predates

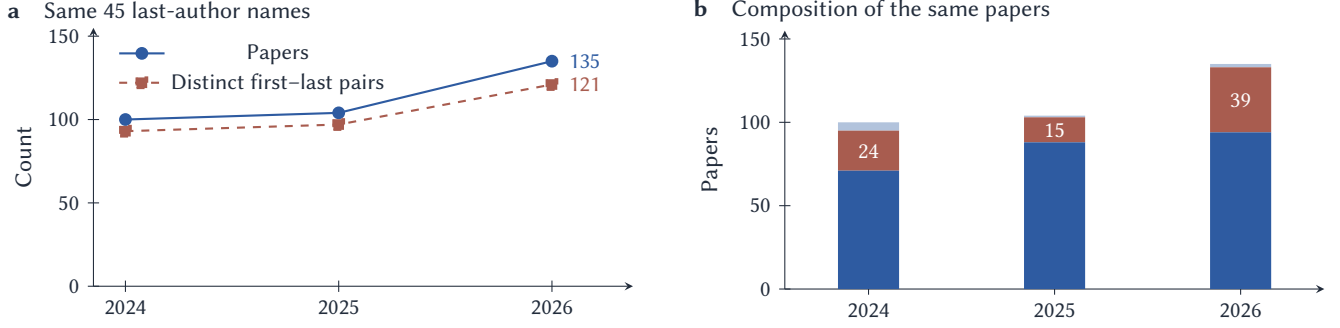


Figure 8: An uneven change in recurring author portfolios. The same 45 last-author names appear in DAC, DATE, and ASP-DAC in all three years. a, Paper counts and distinct first-last pairs. b, Established formulations (blue), new computational formulations (red-brown), and external intervention (light blue). Explicitly LLM-related papers and Late Breaking Results are excluded.

Many investigations can improve one tool

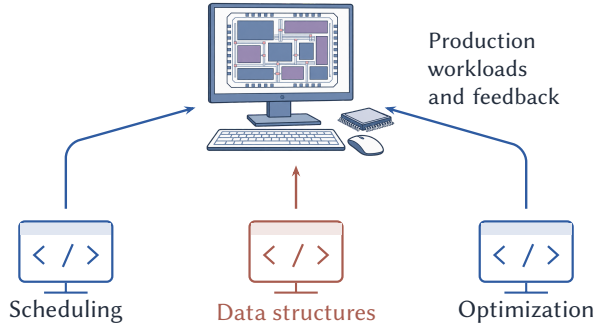


Figure 9: The industrial opportunity includes improvements within existing tools. Cheaper exploration makes more investigations into scheduling, data structures, and optimization worth attempting against production workloads. Small gains can be reused across many designs.

the agent workflows studied here. A lack of a coherent field-wide signal in that population leaves an open question for subsequent cohorts. It does not reduce the value of tracking how the distribution changes.

Counts should be read alongside coverage and concentration. Within each EDA problem, do additional papers explore more constraints and workloads, or repeat a narrower set of method-benchmark combinations? Messeri and Crockett discuss scientific monocultures associated with AI[27]; Hao et al. distinguish individual gains from changes in collective scientific focus[28]. Work on attention in large literatures provides another reason to examine what gets reused as output expands[56]. Conversely, systems designed to seek less-attended questions illustrate how search could broaden the agenda[55].

These observations are possible with public records (Fig. 10). Compare combinations against shuffled assignments that preserve their ingredients’ popularity[36]. Keep first- and last-author positions separate to distinguish new partnerships from repeated output

within the same pairs. Link preprints to later venue versions, and use first public or submission dates where available. Changing EDA preprint habits can then be studied alongside changes in research production.

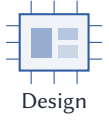
The observations are most informative together. Rising output with wider problem coverage describes a different trajectory from rising output with repeated methods and benchmarks. Faster diffusion between otherwise separate groups is another change worth following. Such patterns can help the community see where cheaper exploration is opening new work and where shared evaluators, broader workloads, or a different choice of questions would be useful.

6 CONCLUSION

We deliberately explored unfamiliar EDA questions and found that substantial algorithmic work could proceed with little continuous specialist input. Some results became useful tools; others failed their practical comparison. Cheaper exploration offers tool developers more opportunities to improve their systems. For academic EDA, it also raises an open question: what should we contribute beyond increasingly inexpensive algorithm search, and how should publication, hiring, promotion, and funding recognize that value?

DATA AND RESEARCH MATERIALS

We assembled the paper-level corpus and semantic labels, author-role summaries, tag analyses, figure data and reproduction scripts in the companion materials, together with the ART project reports and supplied research archives. Process data identify the timestamps, ledger categories, and selected intervention episodes used here; the weekly human-time estimate is retrospective. The analyses preserve the three closure categories, distinguish primary and expanded core EDA, exclude Late Breaking Results, and retain accepted Early Access papers. The manuscript snapshot is 7 October 2026. Seven associated ART manuscripts are available as preprints[1–7].

a Problem coverage

Design



Methods

Counts and shares by problem.
Which research loops expand?

b Repeated combinations

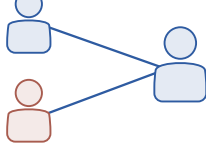
Method

Benchmark

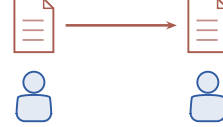
Method

Benchmark

Recurring combinations
within each problem.

c Collaboration structure

First and last authors.
New partners or
repeated pairs?

d Diffusion between groups

Time until a combination
reaches another group.

Figure 10: Public-data observations for subsequent EDA cohorts. Track coverage, recurring methods and benchmarks, collaboration, and diffusion together; link preprints to their later venue versions.

AI USE DISCLOSURE

We used multiple versions of OpenAI GPT models, Kimi, and DeepSeek in the research and preparation of this article. All manuscript prose was generated by AI. Humans provided the initial initiative, checked the material, and adjusted the overall direction; the trial design sought to minimize human intervention in the specific algorithms, with the interventions described in the text. AI was also used for literature retrieval, semantic annotation, analysis code, and figure preparation. Human authors are responsible for the article’s claims and citations.

REFERENCES

- [1] K. Zhu. Provably Good Prim–Dijkstra Revisited: New Theory and a Practical Algorithm for a Classical VLSI Routing Problem with LLMs. 2026. arXiv:2607.17005.
- [2] K. Zhu. Rethinking Logic Optimization Operators: Theory-Derived Operator Compression via Agentic Source Analysis. 2026. arXiv:2607.23672.
- [3] P. Su. Batch Before You Time: Decision-Scoped Proxy Execution for Timing-Aware Logic Rewriting. 2026. arXiv:2609.02470.
- [4] Y. Deng, X. Yang, and K. Zhu. Safe Hypergraph Contraction via Capacity-Aware Repair Certificates. 2026. arXiv:2610.01678.
- [5] B. Song and K. Zhu. Beyond No-Good Benders Cuts: Exact Realizability for Discretely Tunable Clock Trees. 2026. arXiv:2610.07600.
- [6] Z. Jiang, H. Pan, C. Lan, and K. Zhu. Timing-Driven Logic Remapping with Local Physical Context. 2026. arXiv:2610.01918.
- [7] X. Hao and K. Zhu. FAPO: Fanout-Aware Post-Mapping Optimization for LUT-Based FPGAs. 2026. arXiv:2610.06937.
- [8] B. Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi:10.1038/s41586-023-06924-6.
- [9] A. Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. 2025. arXiv:2506.13131.
- [10] B. Georgiev, J. Gómez-Serrano, T. Tao, and A. Z. Wagner. Mathematical exploration and discovery at scale. 2025. arXiv:2511.02864.
- [11] M. Abouzaid, N. Srivastava, R. Ward, and L. Williams. First Proof Second Batch. 2026. arXiv:2606.18119.
- [12] E. Moskowitz. First Proof’s second batch of math problems test AI. *Harvard FAS Current*, 17 June 2026. <https://current.fas.harvard.edu/stories/first-proofs-second-batch-math-problems-test-ai>.
- [13] T. Tao. Mathematics in the age of AI. 2026. arXiv:2608.16753.
- [14] C. Lu et al. Towards end-to-end automation of AI research. *Nature*, 651:914–919, 2026. doi:10.1038/s41586-026-10265-5.
- [15] J. Blocklove, S. Garg, R. Karri, and H. Pearce. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. *MLCAD*, 2023. doi:10.1109/MLCAD58807.2023.10299874.
- [16] M. Liu et al. ChipNeMo: Domain-Adapted LLMs for Chip Design. 2023. arXiv:2311.00176.
- [17] H. Wu et al. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. *IEEE TCAD*, 2024. doi:10.1109/TCAD.2024.3383347.
- [18] Z. He, Y. Pu, H. Wu, T. Qiu, and B. Yu. Large Language Models for EDA: Future or Mirage? *ACM TODAES*, 2025. doi:10.1145/3736167.
- [19] R. Brayton and A. Mishchenko. ABC: An Academic Industrial-Strength Verification Tool. *CAV*, pp. 24–40, 2010. doi:10.1007/978-3-642-14295-6_5.
- [20] T. Ajayi et al. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. *DAC*, 2019. doi:10.1145/3316781.3326334.
- [21] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. *DAC*, 2019. doi:10.1145/3316781.3317803.
- [22] Design Automation Conference. Research topics. *DAC 2027*. Accessed 7 October 2026. <https://dac.com/2027/program/research-topics>.
- [23] Design Automation Conference. Research manuscript submissions. *DAC 2026*. <https://dac.com/2026/research-manuscript-submissions>.
- [24] I. M. Cockburn, R. Henderson, and S. Stern. The Impact of Artificial Intelligence on Innovation. *NBER Working Paper* 24449, 2018. doi:10.3386/w24449.
- [25] N. Bloom, C. I. Jones, J. Van Reenen, and M. Webb. Are Ideas Getting Harder to Find? *American Economic Review*, 110(4):1104–1144, 2020. doi:10.1257/aer.20180338.
- [26] C. Si, D. Yang, and T. Hashimoto. Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers. *ICLR*, 2025. arXiv:2409.04109.
- [27] L. Messeri and M. J. Crockett. Artificial intelligence and illusions of understanding in scientific research. *Nature*, 627:49–58, 2024. doi:10.1038/s41586-024-07146-0.

- [28] Q. Hao, F. Xu, Y. Li, and J. Evans. Artificial intelligence tools expand scientists' impact but contract science's focus. *Nature*, 649:1237–1243, 2026. doi:10.1038/s41586-025-09922-y.
- [29] IEEE Council on Electronic Design Automation. Board of Governors meeting minutes, June 2025. https://ieee-ceda.org/files/ieeeceda/2025-08/CEDA_BoG_Minutes_June_2025_final.pdf.
- [30] D. Chen. ICCAD'25 has set new records: 1,078 submissions, 608 attendees, and representation from 34 countries. *LinkedIn*, 2025. https://www.linkedin.com/posts/demingchen_iccad25-has-set-new-records-1078-submissions-activity-7389672260082704384-5mhO.
- [31] T. Sato. Message from the Technical Program Committee. *ASP-DAC 2026 Full Program*, p. 2, 2026. https://www.aspdac.com/aspdac2026/pdf/ASP-DAC_2026_Full_Program.pdf.
- [32] Zhihu contributors. How should I view the review comments for DAC 2025? *Zhihu*, 2025. <https://www.zhihu.com/en/answer/84984217642>.
- [33] A. Korinek. AI Agents for Economic Research. August 2025. <https://www.aeaweb.org/content/file?id=23290>.
- [34] K. Harris. Mass-produced science is coming. What happens to scientists? *The Transmitter*, 9 July 2026. doi:10.53053/MVLT4623.
- [35] M. D. Schwartz. Claude-shaped science. *Anthropic*, 1 October 2026. <https://www.anthropic.com/research/claude-shaped-science>.
- [36] B. Uzzi, S. Mukherjee, M. Stringer, and B. Jones. Atypical Combinations and Scientific Impact. *Science*, 342(6157):468–472, 2013. doi:10.1126/science.1240474.
- [37] H. Wang et al. Scientific discovery in the age of artificial intelligence. *Nature*, 620:47–60, 2023. doi:10.1038/s41586-023-06221-2.
- [38] A. Davies et al. Advancing mathematics by guiding human intuition with AI. *Nature*, 600:70–74, 2021. doi:10.1038/s41586-021-04086-x.
- [39] A. Fawzi et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53, 2022. doi:10.1038/s41586-022-05172-4.
- [40] D. J. Mankowitz et al. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618:257–263, 2023. doi:10.1038/s41586-023-06004-9.
- [41] K. E. Murray et al. VTR 8: High-Performance CAD and Customizable FPGA Architecture Modelling. *ACM Transactions on Reconfigurable Technology and Systems*, 13(2), Article 9, 2020. doi:10.1145/3388617.
- [42] G.-J. Nam, C. J. Alpert, P. Villarrubia, B. Winter, and M. C. Yildiz. The ISPD2005 placement contest and benchmark suite. *ISPD*, pp. 216–220, 2005. doi:10.1145/1055137.1055182.
- [43] G.-J. Nam, C. J. Alpert, and P. G. Villarrubia. The ISPD 2006 Placement Contest and Benchmark Suite. *ISPD*, contest presentation, 2006. <https://www.ispd.cc/slides/2006/7-3.pdf>.
- [44] L. Amarù, P.-E. Gaillardon, and G. De Micheli. The EPFL Combinational Benchmark Suite. *International Workshop on Logic & Synthesis*, 2015.
- [45] A. Basak Chowdhury, B. Tan, R. Karri, and S. Garg. OpenABC-D: A Large-Scale Dataset for Machine Learning Guided Integrated Circuit Synthesis. 2021. arXiv:2110.11292.
- [46] M. Liu, N. Pinckney, B. Khailany, and H. Ren. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. *ICCAD*, 2023. arXiv:2309.07544.
- [47] J. Lu et al. ePlace: Electrostatics-Based Placement Using Fast Fourier Transform and Nesterov's Method. *ACM TODAES*, 20(2), Article 17, 2015. doi:10.1145/2699873.
- [48] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang. RePlAce: Advancing Solution Quality and Routability Validation in Global Placement. *IEEE TCAD*, 38(9):1717–1730, 2019. doi:10.1109/TCAD.2018.2859220.
- [49] A. Mirhoseini et al. A graph placement methodology for fast chip design. *Nature*, 594:207–212, 2021. doi:10.1038/s41586-021-03544-w.
- [50] C.-K. Cheng, A. B. Kahng, S. Kundu, Y. Wang, and Z. Wang. An Updated Assessment of Reinforcement Learning for Macro Placement. *IEEE TCAD*, 2026. doi:10.1109/TCAD.2025.3644293.
- [51] S. Fortunato et al. Science of science. *Science*, 359(6379):eaao0185, 2018. doi:10.1126/science.aao0185.
- [52] L. Wu, D. Wang, and J. A. Evans. Large teams develop and small teams disrupt science and technology. *Nature*, 566:378–382, 2019. doi:10.1038/s41586-019-0941-9.
- [53] D. Kobak, R. González-Márquez, E.-Á. Horvát, and J. Lause. Delving into LLM-assisted writing in biomedical publications through excess vocabulary. *Science Advances*, 11(27):eadt3813, 2025. doi:10.1126/sciadv.adt3813.
- [54] B. F. Jones. The Burden of Knowledge and the “Death of the Renaissance Man”: Is Innovation Getting Harder? *Review of Economic Studies*, 76(1):283–317, 2009. doi:10.1111/j.1467-937X.2008.00531.x.
- [55] J. Sourati and J. A. Evans. Accelerating science with human-aware artificial intelligence. *Nature Human Behaviour*, 7:1682–1696, 2023. doi:10.1038/s41562-023-01648-z.
- [56] J. S. G. Chu and J. A. Evans. Slowed canonical progress in large fields of science. *Proceedings of the National Academy of Sciences*, 118(41):e2021636118, 2021. doi:10.1073/pnas.2021636118.
- [57] W. Liang et al. Monitoring AI-Modified Content at Scale: A Case Study on the Impact of ChatGPT on AI Conference Peer Reviews. *ICML*, Proceedings of Machine Learning Research 235:29575–29620, 2024. <https://proceedings.mlr.press/v235/liang24b.html>.
- [58] G. Dong, Y. Gao, L. Li, T. Xu, Y. Hua, and F. Yang. Agent Skills Can Be Harmful: An Empirical Study of Skill-Induced Failures in LLM Agents. 2026. arXiv:2608.11888.
- [59] Z. Chen, Z. Sun, Y. Shi, C. Peng, X. Gu, D. Lo, and L. Jiang. Rethinking the Value of Agent-Generated Tests for LLM-Based Software Engineering Agents. 2026. arXiv:2602.07900v2.